

Отчет АЗ

Обе реализации обладают теоретической асимптотикой $O(n \cdot \log n)$, однако существенно различаются по величине скрытых констант, числу рекурсивных вызовов и устойчивости к неблагоприятным входным данным.

Результаты измерений

1. Массивы со случайными значениями

Анализ графиков показывает, что:

- Кривые стандартного Quick Sort и гибридного Introsort располагаются очень близко друг к другу.
- Для случайных входов Quick Sort почти всегда создаёт хорошо сбалансированные разбиения, что минимизирует глубину рекурсии.
- У Introsort иногда наблюдаются локальные скачки времени, связанные с переходом на Heap Sort.
- В среднем Quick Sort работает чуть стабильнее и иногда даже быстрее, благодаря отсутствию дополнительных проверок глубины и переключений.

На случайных массивах Introsort не быстрее стандартного Quick Sort.

Они близки по времени, но у Introsort заметны скачки, так как он иногда переключается на Heap Sort.

2. Обратно отсортированные массивы

Здесь всё наоборот:

- Quick Sort хуже адаптируется и даёт значительно больший рост времени.
- Introsort идёт намного ниже и явно быстрее. Он рано достигает глубины рекурсии $2 \cdot \log n$, переключается на Heap Sort и избегает квадратичного сценария.

На обратно отсортированных данных Introsort значительно быстрее и стабильнее.

Преимущество достигает десятков процентов благодаря защите от неблагоприятных разбиений.

3. Почти отсортированные массивы

Здесь преимущество гибридного алгоритма проявляется ещё ярче:

- Стандартный Quick Sort демонстрирует скачки времени — он нередко попадает в неудачные разбиения из-за структуры входных данных.
- Introsort работает гораздо ровнее.
- Включение Insertion Sort на малых подмассивах даёт ощутимый выигрыш, так как алгоритм имеет почти линейное время на почти отсортированных данных.
- Наблюдается устойчивое снижение времени выполнения по сравнению с классическим Quick Sort.

На почти отсортированных массивах Introsort заметно превосходит стандартный Quick Sort иногда на 25–35 %.

Выводы

- Стандартный Quick Sort работает очень быстро на случайных данных, но подвержен нестабильности и может давать всплески времени на упорядоченных массивах
- Introsort
 - стабильно эффективнее на почти отсортированных и обратно отсортированных массивах
 - полностью предотвращает квадратичное время выполнения
 - демонстрирует существенно меньшие скачки времени
 - в среднем даёт ровную и предсказуемую производительность
- На случайных данных Introsort не быстрее Quick Sort, поскольку дополнительные проверки и переключения не дают выигрыша и иногда создают небольшие накладные расходы.
- Оба алгоритма имеют сложность $O(n \log n)$, но Introsort обладает:
 - большей устойчивостью,
 - меньшей чувствительностью к структуре входного массива
 - более плавной зависимостью времени выполнения от n

ID отправки на CodeForces: [348592333](https://codeforces.com/contest/1462/submission/348592333)